



Tecnológico de Monterrey



Construcción de Software y Toma de Decisiones (Gpo 401)

Actividad:

Avance de Proyecto 4: Prueba de concepto

Profesores:

Ricardo Cortés Espinosa | Eduardo Juárez Pineda

Empresa:

PPG Industries, Inc.

Socios formadores:

Jorge Alberto Alba Tapia | Ariel Lugo Albor

Integrantes:

Cristhian Viery Maida Suarez - A01668790

José Eduardo Viveros Escamilla - A01710605

Laura Cintora Cendejas - A01712379

Gerardo Martínez Carbajal - A01713474

Construcción de Software y Toma de Decisiones (Gpo 401).....	1
Avance de Proyecto 4: Prueba de concepto.....	1
1. Avance del 90% del análisis de requisitos funcionales.....	3
3. Avance del 30% del diseño e implementación de requisitos funcionales.....	6
4. Diseño y ejecución de pruebas:.....	8

1. Avance del 90% del análisis de requisitos funcionales.

En este avance 4 se consolidó el análisis funcional del sistema de las preventas para la empresa PPG, donde se incorporaron todos los ajustes que son derivados de la retroalimentación de los socios formadores y profesores.

Se mantuvieron los 24 requisitos funcionales identificados y priorizados; como resultado, el análisis funcional presenta un avance cercano al 90%, es decir, 22 requisitos están definidos. En esta fase, se fortaleció su redacción, trazabilidad y consistencia, asegurando su alineación con las reglas de negocio, los requisitos no funcionales y los reportes contemplados dentro del sistema. El proyecto toma como base a una tabla de priorización que es evaluada mediante valor, dificultad, riesgo y estabilidad. Para esta entrega se toma como base el bloque de mayor prioridad, conformado por los casos CU-01 a CU-09, debido a que concentran la operación principal del sistema.

Los 9 casos de alta prioridad:

- Concesionario registra producto en carrito de preventa.
- Concesionario consulta catálogo de productos de la campaña activa.
- Concesionario selecciona cuenta activa antes de operar en el sistema.
- Concesionario modifica productos previamente agregados al carrito antes de confirmarla.
- Concesionario registra confirmación de reserva generando un folio único auditable.
- Administrador registra productos (SKU) en el catálogo de campaña.
- Administrador configura campaña activa (nombre, fechas, banner y temporizador).
- Administrador consulta bitácora de auditoría filtrada por usuario y periodo.

Todos los casos de uso detallados están homologados del mismo formato válido del CU-01: descripción, nombre, actores, precondiciones, postcondiciones, flujo básico, flujos alternativos, condiciones especiales, diagrama de actividad, árbol de decisiones, diagrama de secuencia y tabla de pruebas; esto nos permite mantener trazabilidad y consistencia documental.

2. Descripción inicial de la arquitectura de software

La estructura general de la aplicación y su(s) estilo(s) arquitectónico(s).

Esta aplicación es un sistema web monolítico basado en Node.js, Express y EJS. Su punto de entrada son las app.js, donde se configura el servidor, la sesión, la seguridad básica, las rutas y el motor de vistas. El estilo arquitectónico principal es MVC del lado del servidor, con una organización por capas. A continuación mostramos cómo tenemos implementada nuestra arquitectura:

Shell

Fourware_systems/

```
|— app.js # Punto de entrada de la aplicación
|— package.json # Dependencias y scripts
|— config/
|   └─ db.js # Configuración de conexión a MySQL
|— middlewares/
|   └─ auth.middleware.js # Autenticación y autorización
|— routes/
|   └─ auth.routes.js # Rutas de login/logout
|   └─ admin.routes.js # Rutas del módulo administrador
|   └─ logistic.routes.js # Rutas del módulo logística
|   └─ marketing.routes.js # Rutas del módulo marketing
|   └─ concesionario.routes.js # Rutas del módulo concesionario
|   └─ carrito.routes.js # Rutas del carrito y reservas
|— controllers/
|   └─ auth.controller.js # Lógica de autenticación
|   └─ admin.controller.js # Lógica del administrador
|   └─ logistic.controller.js # Lógica de logística
|   └─ marketing.controller.js # Lógica de marketing
|   └─ concesionario.controller.js # Lógica del concesionario
|   └─ carrito.controller.js # Lógica del carrito/reservas
|— models/
|   └─ usuario.model.js # Acceso a datos de usuarios, roles y
privilegios
|   └─ concesionario.model.js # Consultas del módulo concesionario
|   └─ reserva.model.js # Persistencia de reservas
|   └─ bitacora.model.js # Registro de auditoría
|— views/
|   └─ layouts/
|       └─ main.ejs # Layout principal
|   └─ includes/
|       └─ header.ejs # Encabezado compartido
|       └─ sidebar.ejs # Barra lateral compartida
|       └─ footer.ejs # Pie de página compartido
|   └─ login.ejs # Vista de acceso
|   └─ dashboard.ejs # Dashboard principal
|   └─ modules/ # Vistas del módulo admin y
concesionario
|   └─ marketing/ # Vistas del módulo marketing
|   └─ logistica/ # Vistas del módulo logística
|— public/
|   └─ css/
|       └─ styles.css # Estilos de la aplicación
|   └─ img/ # Recursos estáticos
|— utils/
|   └─ resetPasswords.js # Utilidad de soporte
```

La aplicación se divide en los siguientes componentes funcionales:

Autenticación y sesión

- Gestiona inicio y cierre de sesión.
- Consulta usuarios, contraseñas, roles y privilegios.
- Archivos principales: [routes/auth.routes.js](#), [controllers/auth.controller.js](#), [models/usuario.model.js](#)

Autorización

- Restringe acceso por sesión activa, rol y privilegios.
- Archivo principal: [middlewares/auth.middleware.js](#)

Administración

- Dashboard, catálogo, campañas, reportes y auditoría.
- Archivos principales: [routes/admin.routes.js](#), [controllers/admin.controller.js](#)

Logística

- Reservas confirmadas, métricas y reporte operativo.
- Archivos principales: [routes/logistic.routes.js](#), [controllers/logistic.controller.js](#)

Marketing

- Métricas comparativas, ranking de productos y tendencias por región.
- Archivos principales: [routes/marketing.routes.js](#), [controllers/marketing.controller.js](#)

Concesionario

- Home, catálogo, detalle de producto, consulta y cancelación de reservas.
- Archivos principales: [routes/concesionario.routes.js](#), [controllers/concesionario.controller.js](#), [models/concesionario.model.js](#)

Carrito y reservas

- Agregar productos, visualizar carrito, eliminar productos y confirmar reserva.
- Archivos principales: [routes/carrito.routes.js](#), [controllers/carrito.controller.js](#), [models/reserva.model.js](#)

Bitácora de auditoría

- Registra eventos relevantes del sistema.
- Archivo principal: [models/bitacora.model.js](#)

La aplicación se integra con datos externos principalmente mediante:

Base de datos MySQL

- La conexión se define en config/[db.js](#)
- Los modelos ejecutan consultas SQL directas usando mysql2

- Entidades definidas con las que se conecta nuestra arquitectura: Usuario, Rol, Privilegio, Reserva, Reserva_Producto, Producto, BitacoraAuditoria

HTTP/HTML con el navegador

- La interfaz de usuario se entrega por medio de vistas EJS.
- Las rutas HTTP residen en formulario y devuelven HTML renderizado por el lado del servidor.

Sesión en servidor

- Se usa express-session para almacenar usuario autenticado, roles, privilegios y carritos.
- Esta interfaz soporta el estado temporal entre peticiones.

Recursos estáticos

- CSS e imágenes se sirven desde [public](#).

La descripción de cómo la arquitectura soporta los requisitos no funcionales. Considerando que tenemos lo siguiente:

Seguridad

- Se soporta mediante sesiones, control de acceso por rol y privilegio

Mantenibilidad

- La separación en rutas, controladores, modelos y vistas facilita localizar cambios y extender módulos.
- La organización por dominio funcional ayuda a evolucionar cada área del negocio de forma aislada.

Escalabilidad

- La separación en rutas, controladores, modelos y vistas facilita localizar cambios y extender módulos.

Trazabilidad

- La bitácora permite registrar acciones relevantes para seguimiento operativo y control.

3. Avance del 30% del diseño e implementación de requisitos funcionales.

En esta etapa se presenta un avance parcial del diseño e implementación de la solución. En el repositorio del equipo https://github.com/Eduardo0198/Fourware_systems se puede visualizar su integración de la estructura técnica base del sistema bajo una organización modular, con separación entre configuración, controladores, middlewares, modelos, rutas y vistas.

Esta afirmación se sustenta en la evidencia directa del código fuente, donde ya se encuentran implementados módulos funcionales de autenticación, control de acceso, administración de campañas, catálogo, carrito, reservas y auditoría.

El proceso de autenticación y autorización ya está implementado en el sistema. El inicio de sesión, validación de credenciales y redirección por rol se encuentran en `controllers/auth.controller.js`, mientras que la consulta de usuarios, contraseñas, roles y privilegios se realiza desde `models/usuario.model.js`. Asimismo, la protección de rutas, la validación por rol, la validación por privilegios y la verificación de cuenta activa para concesionarios se encuentran en `middlewares/auth.middleware.js`. Esto demuestra que ya existe implementación funcional de los requisitos relacionados con acceso seguro y controlado al sistema.

En el módulo del concesionario ya se encuentra implementado el flujo principal de operación. La consulta del catálogo, la búsqueda, la paginación y los filtros se gestionan en `controllers/concesionario.controller.js` y en `models/concesionario.model.js`.

La visualización del catálogo y del detalle de producto se refleja en las vistas `views/modules/concesionarioCatalogo.ejs` y `views/modules/concesionarioProducto.ejs`. Además, la selección de cuenta activa y la validación de cuentas asociadas al usuario están implementadas en `controllers/concesionario.controller.js` y `models/cuenta.model.js`, lo cual fortalece la lógica de negocio propia del proceso comercial.

El carrito de preventa es uno de los componentes con mayor evidencia de implementación. Las operaciones de agregar productos, consultar carrito, actualizar cantidades, eliminar productos y confirmar reservas están desarrolladas en `controllers/carrito.controller.js`. En este módulo ya se observa lógica de negocio real, como validación de campaña activa, control de cantidades válidas, cálculo de subtotal, IVA, total, peso y volumen, validación de sucursal y generación de folio de reserva. La persistencia de la reserva y sus productos se realiza mediante transacción en `models/reserva.model.js`, mientras que la interfaz de esta funcionalidad se encuentra en `views/modules/concesionarioCarrito.ejs`.

En el módulo administrativo también existe evidencia sólida de diseño e implementación. La consulta del catálogo administrativo, el registro de productos, la creación y edición de campañas, la activación exclusiva de campañas, la desactivación y la configuración de la ventana de cancelación están implementadas en `controllers/admin.controller.js`. La lógica de persistencia para estas operaciones se encuentra en `models/producto.model.js`, `models/campania.model.js` y `models/cancelacion.model.js`. Como evidencia visual del avance, se cuenta con las vistas

`views/modules/adminCatalogo.ejs,` `views/modules/campanaCrear.ejs,`
`views/modules/campanaEditar.ejs` y `views/modules/campanaCancelacion.ejs.`

Otro elemento importante ya implementado es la auditoría del sistema. El registro de eventos se concentra en `utils/auditoria.helper.js` y se almacena mediante `models/bitacora.model.js`. Además, la consulta administrativa de la bitácora, incluyendo filtros por usuario y por fechas, ya se encuentra funcionalmente contemplada en `controllers/admin.controller.js` y se representa visualmente en `views/modules/adminAuditoria.ejs`. Esta parte del sistema aporta trazabilidad y control sobre los eventos relevantes del uso de la aplicación.

Como parte de la evidencia técnica de esta sección, también se entrega el script base de la estructura de base de datos en `database/schema.sql`, construido a partir de las entidades, relaciones y consultas identificadas en el código. La conexión actual del sistema a MySQL se encuentra definida en `config/db.js`, lo cual demuestra que la aplicación ya opera sobre una base persistente y no únicamente sobre un prototipo visual.

Sin embargo, también se identifican funcionalidades que todavía no se consideran terminadas y que permanecen en estado de prototipo, diseño parcial o implementación incompleta. Entre ellas se encuentran la modificación completa de productos, la carga masiva, el historial funcional de reservas, el detalle real de reservas, la cancelación de reservas, así como la generación de reportes y las consultas reales para los módulos de marketing y logística. Esto puede observarse en vistas como `views/modules/catalogoModificar.ejs,` `views/modules/catalogoCargaMasiva.ejs,` `views/modules/concesionarioReservas.ejs,` `views/modules/concesionarioDetalleReserva.ejs,` `views/marketing/metricasComparativas.ejs` y `views/logistica/reservasConfirmadas.ejs,` donde aún no se aprecia una integración completa con lógica de negocio y datos reales.

4. Diseño y ejecución de pruebas:

Esta parte de las pruebas del proyecto parte de la traducción formal de casos de uso a casos de prueba, con el objetivo de que cada funcionalidad esté definida en el análisis funcional, donde pueda verificarse de manera clara y objetiva durante la operación del sistema.

Como parte de este avance, se cuenta con un repositorio compartido en Drive que concentra la documentación de los casos de uso del proyecto y sus respectivos formatos de prueba. Este material funciona para respaldar nuestro trabajo y la evidencia del seguimiento del sistema.

Para esta etapa en particular, se trabajó con 9 (nueve) casos de uso priorizados, los cuales corresponden al porcentaje de cobertura solicitado para el avance. A continuación, se presenta el enlace al repositorio donde se encuentran de forma organizada los documentos correspondientes:

https://drive.google.com/drive/folders/1aX17bsL8et1hHCNj2HYz2Ad0CCTooTpq?usp=drive_link

En esta etapa se avanzó en el diseño y ejecución de pruebas para 9 casos de uso priorizados. En cada prueba se verificó el comportamiento del sistema por medio de diferentes escenarios, desde aquellos en los que existe una posibilidad dentro del límite y fuera. Asimismo, el diseño de pruebas permitió identificar áreas de mejora tanto en la lógica funcional como en la interacción con la interfaz.

Los resultados obtenidos sirvieron para realizar ajustes y correcciones oportunas en aquellos casos donde se detectaron inconsistencias; esto nos permite estar reforzando la confiabilidad del sistema por medio del comportamiento observado durante su ejecución. Por ello esta sección documenta las pruebas y su utilidad como mecanismo de validación y mejora continua del proyecto.

5. Usabilidad

La interfaz del sistema de preventa fue diseñada considerando principios de usabilidad orientados a facilitar la operación de los distintos tipos de usuario del sistema: administrador, concesionario, marketing y logística. Aunque el sistema aún se encuentra en evolución, ya es posible identificar evidencia de diseño centrado en el usuario, especialmente en los módulos con mayor nivel de implementación, como autenticación, catálogo, carrito, campañas y bitácora de auditoría.

Para esta etapa del proyecto, la usabilidad se evaluó tomando como referencia las “8 reglas de oro” de Shneiderman, además de los ajustes sugeridos por el cliente y por la revisión heurística realizada por compañeros. El resultado de esta evaluación permitió identificar fortalezas de la interfaz, así como oportunidades de mejora que ya fueron incorporadas en el prototipo funcional.

5.1. Aplicación de las 8 reglas de oro

5.1.1. Buscar consistencia

El sistema mantiene una estructura visual consistente entre pantallas. Se utilizan elementos repetidos como encabezado superior, barra lateral, estilos de botones, mensajes de retroalimentación y organización general de contenidos. Esta consistencia facilita que el usuario reconozca rápidamente cómo navegar entre módulos y qué tipo de acciones puede realizar en cada pantalla.

- Uso de un mismo layout general para todas las vistas protegidas
- Menú lateral con secciones por rol
- Mensajes visuales homogéneos para éxito, advertencia y error
- Formularios con estructura similar en campañas, catálogo y carrito

5.1.2. Permitir atajos para usuarios frecuentes

Aunque el sistema todavía no cuenta con atajos de teclado, sí ofrece accesos rápidos a funciones frecuentes por medio de la navegación visible y contextual. En el caso del concesionario, el encabezado permite cambiar de cuenta activa rápidamente y acceder al carrito sin salir del flujo de trabajo. Para el administrador, la barra lateral permite entrar directamente a catálogo, campañas, reportes y bitácora.

- Acceso directo al carrito desde el encabezado del concesionario.
- Cambio rápido de cuenta activa desde el menú desplegable.
- Navegación inmediata entre módulos mediante sidebar

5.1.3. Ofrecer retroalimentación informativa

El sistema proporciona retroalimentación clara después de cada operación importante. Esto permite al usuario saber si una acción fue realizada correctamente o si ocurrió un error. Esta regla se cumple especialmente en los módulos de campañas, productos, carrito, login y validación de cuenta activa.

- Mensaje de error cuando el login falla.
- Mensaje de éxito al registrar una campaña
- Advertencia al intentar operar sin cuenta activa
- Mensaje de confirmación al agregar un producto al carrito.
- Mensaje de éxito al confirmar una reserva con folio generado.

5.1.4. Diseño de diálogos que generen un cierre.

Los flujos funcionales implementados conducen a una conclusión clara. Por ejemplo, registrar un producto o una campaña termina con un mensaje de éxito y una redirección. En el carrito, el flujo concluye con la confirmación de la reserva y la limpieza del carrito. Esto ayuda al usuario a percibir que su tarea fue completada correctamente.

- Alta de SKU con confirmación de registro.
- Configuración de campaña con mensaje de cierre exitoso.
- Confirmación de reserva con folio visible al finalizar.

5.1.5. Prevenir errores

La interfaz y la lógica del sistema incorporan múltiples validaciones para evitar errores antes de que el usuario complete una operación inválida. Esto reduce reprocesos y mejora la confianza del usuario durante el uso del sistema.

- Validación de cantidad mayor a cero en carrito.

- Validación de fechas en campañas
- Bloqueo de operación si no existe campaña activa.
- Bloqueo de selección de cuentas inactivas
- Validación obligatoria de sucursal antes de confirmar reserva.
- Validación de SKU duplicado al registrar productos.

5.1.6. Permitir reversión sencilla de acciones

El sistema ofrece reversibilidad parcial en acciones operativas. Por ejemplo, en el carrito el usuario puede aumentar, disminuir o eliminar productos antes de confirmar una reserva. También puede cambiar la cuenta activa si seleccionó una incorrecta. Aún existen funciones pendientes para fortalecer esta regla, como la cancelación funcional de reservas ya confirmadas.

- Eliminación de productos del carrito
- Modificación de cantidades
- Cambio de cuenta activa

5.1.7. Mantener al usuario en control.

La interfaz permite que el usuario tome decisiones de forma explícita y visible, en lugar de ejecutar acciones ocultas o automáticas sin contexto. Esto es especialmente importante en los módulos del concesionario, donde el usuario decide con qué cuenta operar, qué productos agregar y en qué sucursal confirmar su reserva.

- Selección explícita de cuenta activa
- Selección manual de sucursal al confirmar reserva
- Filtros configurables en catálogo
- Activación o desactivación manual de campañas en administración

5.1.8. Reducir la carga de memoria de corto plazo

La interfaz busca mostrar la información relevante dentro de la misma pantalla para que el usuario no tenga que recordarla entre pasos. Se presentan elementos contextuales visibles que ayudan a mantener el estado actual del sistema.

- Encabezado con nombre del usuario y cuenta activa.
- Barra promocional con campaña activa visible para concesionario.
- Resumen del carrito con subtotal, IVA, peso y volumen.
- Tablas y tarjetas con información resumida en admin y auditoría.

5.2. Ajustes de usabilidad sugeridos por el cliente

Tomando en cuenta el uso esperado del sistema por parte del cliente, se identifican varios ajustes funcionales y visuales ya incorporados al prototipo, los cuales responden a necesidades reales de operación:

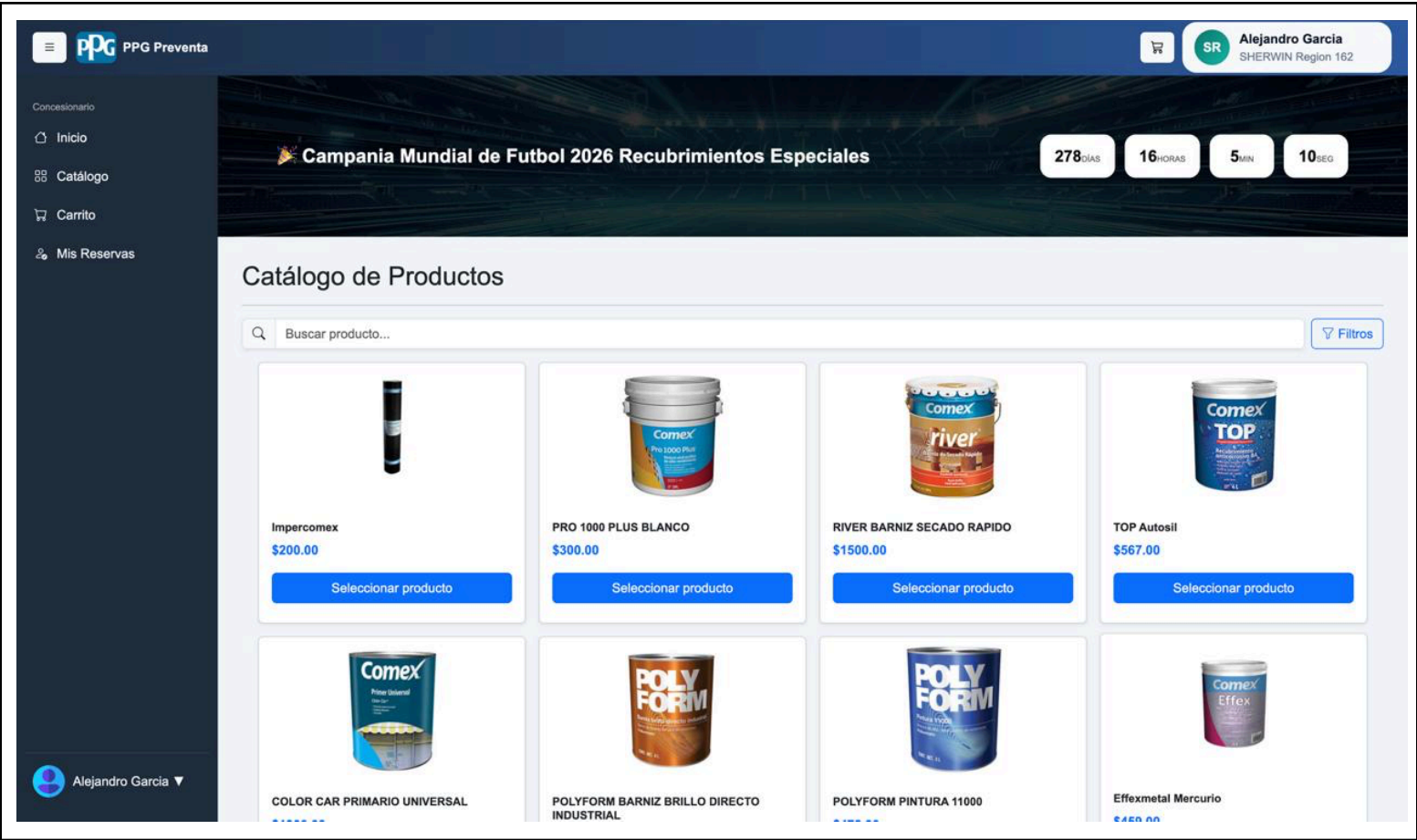
- Visualización clara de la campaña activa en la interfaz del concesionario.
- Separación de funcionalidades por rol para evitar confusión entre usuarios.
- Incorporación de mensajes claros en cada acción importante.
- Posibilidad de seleccionar cuenta activa antes de operar.
- Presentación del resumen del carrito con datos útiles para operación y logística.

Estos ajustes permiten que el sistema sea más entendible para el usuario final y reflejan la intención de adaptar el prototipo a la operación real del negocio.

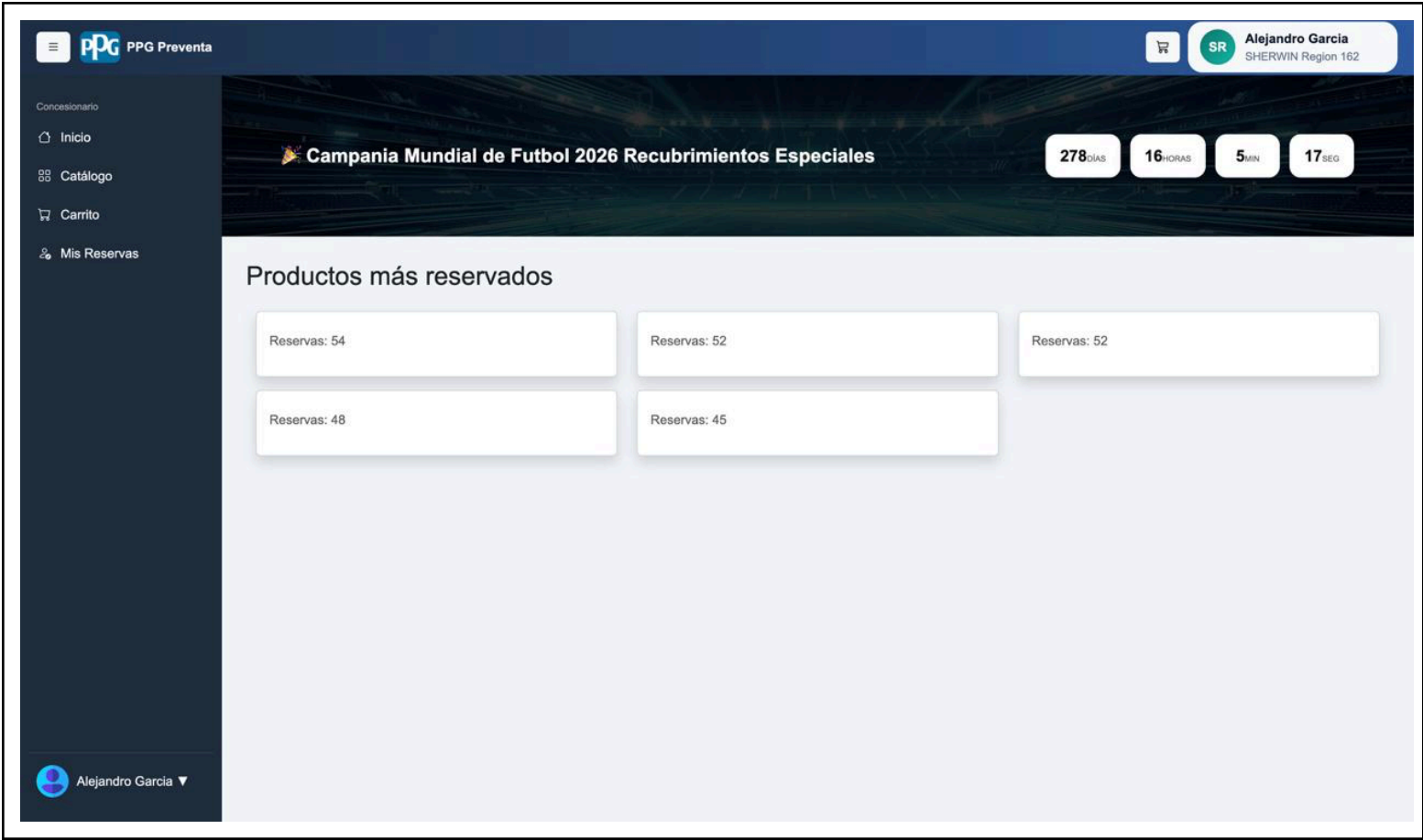
5.3. Mejoras derivadas de la evaluación heurística

Como parte de la evaluación heurística realizada por compañeros, se identificaron oportunidades de mejora en la interfaz. A partir de esas observaciones se realizaron al menos tres mejoras visibles en el prototipo actual.

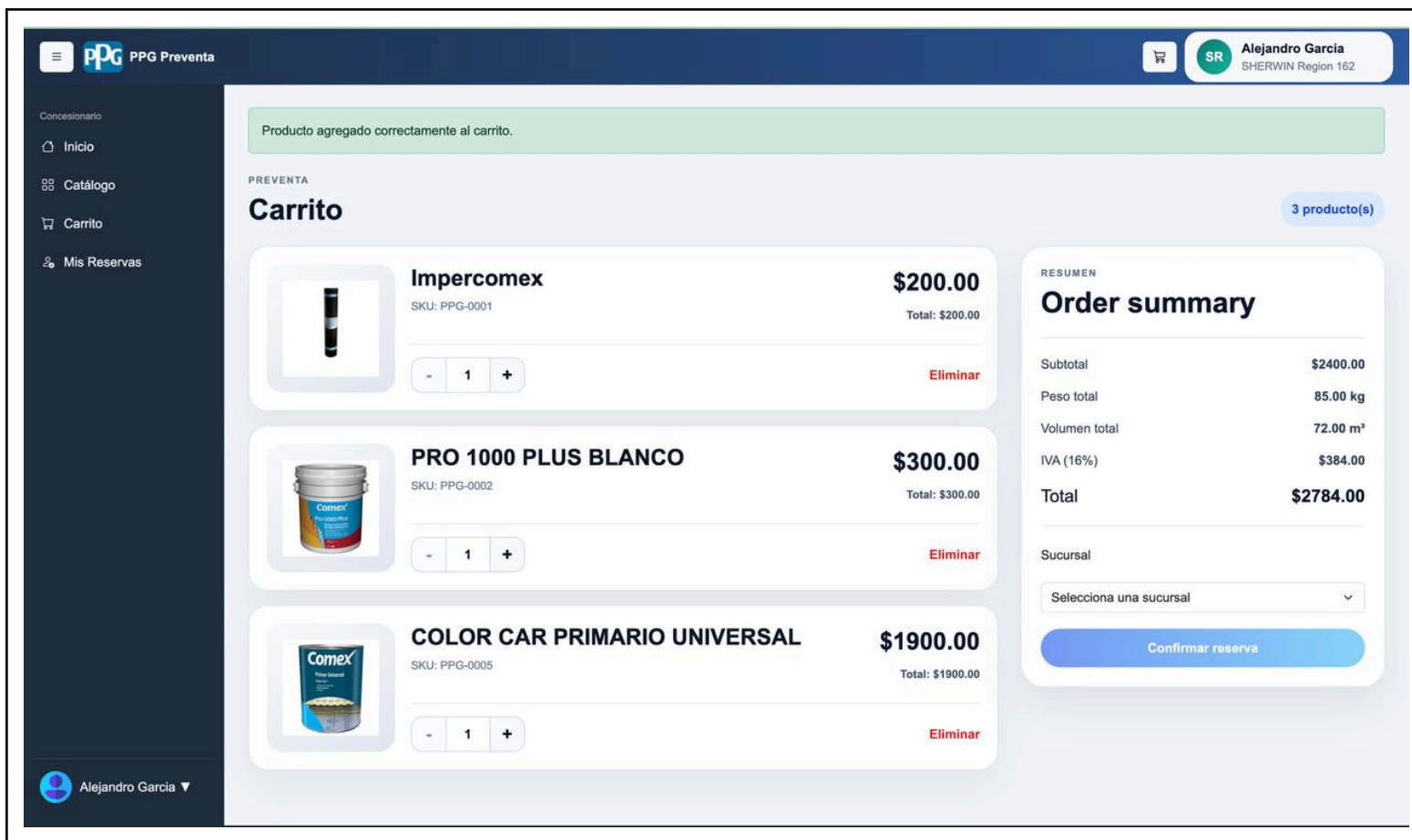
Mejora 1	Vista actual del catálogo de productos con mejoras de usabilidad
Problema detectado:	En versiones anteriores, el usuario no contaba con una vista de catálogo suficientemente clara y estructurada, lo que dificultaba localizar productos, identificar precios y navegar rápidamente entre las opciones disponibles. Además, la operación del concesionario requería mayor visibilidad del contexto actual, como la campaña activa y el acceso rápido a funciones frecuentes.
Mejora aplicada:	Se rediseñó la pantalla del catálogo de productos incorporando una estructura más clara y ordenada. La nueva interfaz muestra un encabezado superior con identidad del sistema, acceso al carrito y datos del usuario, una barra lateral con navegación por módulo, una franja visible de campaña activa con temporizador, un buscador de productos, botón de filtros y tarjetas de producto con imagen, nombre, precio y acción principal de selección.
Beneficio:	Esta mejora incrementa la visibilidad del estado del sistema, facilita la navegación, reduce el tiempo de búsqueda de productos y permite al usuario realizar su proceso de preventa de forma más rápida, clara y segura. Además, mejora la comprensión visual del catálogo y favorece una experiencia más consistente para el concesionario.
Imagen 1:	



Mejora 2	Pantalla de inicio del concesionario con mejora en contexto operativo y navegación
Problema detectado:	La pantalla inicial del concesionario no ofrecía suficiente contexto operativo ni información relevante al ingresar al sistema, por lo que el usuario podía tardar más en ubicarse dentro del módulo y en identificar la campaña vigente o los datos más importantes para comenzar a trabajar.
Mejora aplicada:	La pantalla inicial del concesionario no ofrecía suficiente contexto operativo ni información relevante al ingresar al sistema, por lo que el usuario podía tardar más en ubicarse dentro del módulo y en identificar la campaña vigente o los datos más importantes para comenzar a trabajar.
Beneficio:	La mejora facilita la orientación del usuario dentro del sistema, incrementa la visibilidad del contexto comercial actual y reduce la carga cognitiva al mostrar desde el inicio información relevante para la operación. Además, favorece una navegación más rápida hacia las funciones principales del módulo.
Imagen 2:	



Mejora 3:	Información obtenida de la base de datos correctamente.
Problema detectado:	Antes solo podíamos ver productos que creamos y no estaban siendo obtenidos de la base de datos, en este caso de los productos, y es por eso que no se podían hacer los cálculos de subtotal y total.
Mejora aplicada:	Mostrar datos útiles para el transporte de los productos, es decir, informar sobre el peso total y volumen total para consideración de su futuro transporte.
Beneficio	Mejor planeabilidad a los concesionarios para gestionar su almacenamiento y el transporte de los productos.
Imagen 3:	



5.4. Evidencia de prototipo funcional

El prototipo funcional ya muestra evidencia de varias decisiones orientadas a la usabilidad. Entre las pantallas con mayor madurez visual y funcional se encuentran:

- ◆ login
- ◆ Catálogo del concesionario
- ◆ Detalle de producto
- ◆ carrito
- ◆ Gestión de campañas
- ◆ Alta de productos
- ◆ Bitácora de auditoría

Estas pantallas ya permiten demostrar:

- ◆ Navegación consistente
- ◆ Retroalimentación de acciones
- ◆ Validaciones visibles
- ◆ Reducción de errores
- ◆ Mejor organización de la información.

6. Identificar el ambiente de producción

6.1. *Requisitos técnicos del despliegue*

El proveedor debe soportar:

- Node.js
- MySQL
- Variables de entorno
- dominio propio
- HTTPS
- Certificado SSL
- Acceso para despliegue y monitoreo

6.2. *Propuesta recomendada: Railway*

Se propone Railway como opción principal para este proyecto en etapa académica y de validación, por las siguientes razones:

- Soporta despliegue de aplicaciones Node.js
- permite desplegar MySQL dentro del mismo proyecto.
- Maneja variables de entorno fácilmente.
- Ofrece dominio público para pruebas.
- Ofrece certificados SSL automáticos en networking público.
- Reduce carga operativa comparado con administrar un VPS manualmente.

6.3. *Sustento técnico actual*

Con base en la documentación oficial revisada:

- Railway documenta despliegue para Node.js.
- Railway tiene guía oficial para desplegar MySQL y expone variables como `MYSQLHOST`, `MYSQLPORT`, `MYSQLUSER`, `MYSQLPASSWORD` y `MYSQLDATABASE`.
- Railway documenta networking público con HTTPS y certificados SSL automáticos.
- Railway documenta dominio propio y emisión automática de SSL para dominios personalizados.

6.4. *Costeo inicial de referencia*

Con base en la documentación oficial consultada:

- Railway tiene plan Hobby con consumo mínimo de 5 USD mensuales.
- Railway tiene plan Pro con consumo mínimo de 20 USD mensuales.
- El uso final depende de CPU, RAM, almacenamiento y transferencia.

Inferencia:

Para una primera puesta en operación controlada, una configuración pequeña de app Node.js más MySQL puede iniciar en un rango bajo, útil para prototipo funcional, piloto o demostración.

6.5. *Alternativa*

Como alternativa de mayor control:

- DigitalOcean con Droplet Ubuntu + Node.js + MySQL + Nginx + Certbot/Let's Encrypt

Ventajas:

- Más control técnico.
- Mayor cercanía a producción real.

Desventajas:

- Más carga de administración
- Configuración manual de seguridad, backup y monitoreo.

6.6. *Recomendación final*

Para este avance se recomienda proponer:

- **Railway** como ambiente inicial de despliegue
- Dominio institucional o comercial con HTTPS
- Variables de entorno para secretos
- Migración posterior a proveedor con mayor control si el cliente lo requiere.

7. **Plan de trabajo actualizado y aprendizaje**

Durante este avance, el equipo concentró sus esfuerzos en afinar la definición funcional del sistema, fortalecer la trazabilidad entre análisis y diseño, consolidar la propuesta de reportes, estructurar la arquitectura inicial y construir evidencia navegable del prototipo. Este trabajo implicó una coordinación más estrecha entre documentación, modelado y construcción visual, lo cual demandó más tiempo que una simple actividad de redacción, pero también permitió una mejor integración entre entregables.

En cuanto al seguimiento del trabajo, se observó que varias actividades requirieron más tiempo del estimado originalmente, especialmente aquellas relacionadas con ajustes posteriores a la retroalimentación del cliente y profesores, así como la necesidad de alinear requisitos, modelo de datos, pantallas y evidencia funcional. Esta diferencia entre estimación y tiempo real constituye un aprendizaje relevante para la planeación futura del equipo.

El principal aprendizaje adquirido en este avance puede resumirse en cuatro ejes:

- La importancia de documentar con trazabilidad desde etapas tempranas.
- El valor de priorizar funcionalidades por riesgo y valor de negocio.
- La necesidad de que el diseño de interfaz responda a reglas de negocio reales.
- La conveniencia de construir una arquitectura modular desde el inicio para evitar retrabajo en etapas posteriores.

Notion ... Invitado

FOURWARE SYSTEMS

Buscar

Bandeja de entrada

Compartido

FOURWARE SYSTEMS

Última edición 18 mar

Priority	Aa ACTIVIDAD	Status	Nombre	Fecha de Inicio	Fecha de Fin	#	Tiempo Esperado (min.)
Alta	Corrección de Detallado de casos de uso y diagrama de actividades	Done	Cristhian Viery Maida Sua Gerardo Martínez Carbaja José Eduardo Viveros Esc Laura Cintora Cendejas	10 de marzo de 2026	11 de marzo de 2026		
Alta	Detallado de caso de uso 2 y 3	Done	Cristhian Viery Maida Sua	10 de marzo de 2026	19 de marzo de 2026		
Alta	Detallado de caso de uso 4 y 5	Done	Gerardo Martínez Carbaja Laura Cintora Cendejas	10 de marzo de 2026	19 de marzo de 2026		
Media	Detallado de caso de uso 6 y 7	Done	José Eduardo Viveros Esc	10 de marzo de 2026	19 de marzo de 2026		
Media	Detallado de caso de uso 8	Done	Laura Cintora Cendejas	10 de marzo de 2026	19 de marzo de 2026		
Alta	Casos de Prueba 01	Done	Laura Cintora Cendejas Cristhian Viery Maida Sua Gerardo Martínez Carbaja José Eduardo Viveros Esc	9 de marzo de 2026	27 de marzo de 2026		
Media	Diseño y ejecución de casos de prueba faltantes	Done	Laura Cintora Cendejas Cristhian Viery Maida Sua Gerardo Martínez Carbaja José Eduardo Viveros Esc	9 de marzo de 2026	27 de marzo de 2026		
Alta	Redacción de usabilidad	Done	Laura Cintora Cendejas Cristhian Viery Maida Sua Gerardo Martínez Carbaja José Eduardo Viveros Esc	9 de marzo de 2026	27 de marzo de 2026		

